

Problem A. Coprime Array

We only care about the value of $s \bmod x$. The requirement that the absolute values of the answer are at most 10^9 can be ignored at this level, because any solution can be adjusted to satisfy that range.

Factorize x . It is enough to solve the problem modulo each prime power p^k , then combine the solutions using the Chinese Remainder Theorem.

The answer has length 1 if and only if $\gcd(s, x) = 1$.

In all remaining cases, except when s is odd and x is even, a solution of length 2 can be constructed. In the exceptional case, first split off a 1; then a solution of length 3 can be constructed.

In implementation, randomization can even avoid explicit factorization and CRT. The probability that a random trial hits a solution is at least

$$\prod_{p|x} \frac{\max\{1, p-2\}}{p}.$$

Problem B. Square Locator

Since point A lies on the y -axis, its coordinates can be computed directly.

Enumerate whether the square $ABCD$ is oriented counterclockwise or clockwise. Let $B = (x, y)$. The distances from B and C to the origin give two equations, from which x, y can be solved directly. Then simply check whether the obtained coordinates satisfy all conditions.

Problem C. -is-this-bitset-

For every vertex i of depth $d \leq 11$, set

$$a_i = 2^{20-d}.$$

Because the tree is binary, this changes at most $2^{12} - 1 = 4095$ values.

After this modification, the first 12 vertices on every root-to-vertex chain can generate every multiple of $2^9 = 512$ in the range $[0, 2^{21})$. Therefore, for a vertex of depth greater than 11, it is enough to know, for each residue modulo 512, the minimum value that can be formed. This can be computed by a DP in $O(512)$ time.

To reduce memory usage, observe that along a chain the DP array changes by at most $\max\{b_i\} < 2^{21}$. Store only the changed entries, and roll them back when returning from recursion.

Let $V = \max\{b_i\}$ and $k = 5000$. The time complexity is $O(nV/k)$, and the memory complexity is $O(n + V)$.

Problem D. Cycle Game

Treat all cells outside the board boundary as white. Then a nonempty inner cycle exists if and only if there is a cell such that, after treating this cell as white, the white cells have at least two 8-connected components; equivalently, some white cell is not connected to the boundary.

After inserting a black cell, if such a witness cell exists, at least one witness is adjacent to the newly inserted black cell. Thus we can use divide-and-conquer over time with a rollback DSU to maintain connectivity. At every time, enumerate all cells around the newly added cell and test whether the operation should take effect.

The time complexity is $O(nm \log^2(nm))$.

Problem E. Sum of Squares

No analysis was provided for this problem in the original blog post.

Problem F. Alternating Cycle

No analysis was provided for this problem in the original blog post.

Problem G. Touching Grass

If a query is a line, the problem becomes: does this line intersect the upper convex hull of the point set? This can be answered by binary search on the hull.

A segment query can be reduced to line queries with a segment tree: sort the points by x , build a segment tree, and store the convex hull of each node interval.

This uses $O(n \log n + m)$ memory. By answering the queries offline in a divide-and-conquer style, the memory can be reduced to $O(n + m)$.

If the query segments are sorted by slope, the binary searches on hulls can be replaced by two-pointers. The total time complexity becomes $O((n + m) \log n)$.

Problem H. Game Design

Draw a directed edge $i \rightarrow p_i$. The graph can be regarded as a directed graph in which every vertex has indegree and outdegree equal to 1.

Let $f(i, j)$ be the number of ways after processing the first i vertices, with j incoming/outgoing edge endpoints still undetermined and lying among vertices greater than i .

For the edges incident to vertex i , there are four cases.

- Both the incoming edge and the outgoing edge go to the future. Transition to $j + 1$ with multiplicity 1.
- The incoming edge goes to the future and the outgoing edge goes to the past. Transition to j with multiplicity j .
- The incoming edge goes to the past and the outgoing edge goes to the future. This requires $s_i = 1$; transition to j with multiplicity j .
- Both edges go to the past. This requires $s_i = 1$; transition to $j - 1$ with multiplicity j^2 .

The time complexity is $O(n^2)$.

Problem I. Geometry Hacking

No analysis was provided for this problem in the original blog post.

Problem J. Non-Interactive Nim

No analysis was provided for this problem in the original blog post.